EVENT-DRIVEN COMMERCE

Microservices E-commerce Platform

Project Documentation with Screenshots

Architecture	Event-driven microservices
Backend	Golang • gRPC • REST
Frontend	Next.js • TypeScript • Tailwind CSS
Data	PostgreSQL per service
Messaging	RabbitMQ / AMQP
Gateway	api-gtw REST → internal gRPC
Infrastructure	Docker Compose

Prepared from the supplied Event-driven-microservices-ecommerce-master source ZIP. The documentation preserves the project's terminology and architecture described in its README and source tree.

1. Project Overview

The supplied project is an e-commerce application built around distributed systems, microservices and event-driven architecture. The api-gtw service exposes REST endpoints to the web UI and internally communicates with the customer, product and order services through gRPC. For order payment, the order service publishes a message to RabbitMQ and the payment service processes that message asynchronously.

2. Technology Stack

Layer	Technologies
Backend	Golang, PostgreSQL, RabbitMQ, gRPC, REST
Frontend	TypeScript, Next.js, Tailwind CSS
Communication	REST/HTTP from web UI to gateway; gRPC between gateway and services
Async messaging	RabbitMQ using AMQP for payment processing flow
Deployment/local environment	Docker and Docker Compose

3. Microservices

The supplied source tree contains the following application services:

• **api-gtw:** REST API gateway with HTTP controllers, routers and gRPC clients for customer, order and product services.

• **customer-svc:** customer domain, signup/signin/get operations, password hashing and JWT token generation.

• **product-svc:** product retrieval and quantity update use cases with PostgreSQL/in-memory repository implementations.

• **order-svc:** order creation/retrieval and product-service integration through gRPC.

• **payment-svc:** asynchronous order/payment processing with RabbitMQ queue consumers and PostgreSQL persistence.

• **web-ui:** Next.js application with authentication, products, cart, checkout and order pages.

4. Data & Infrastructure

• Each business service is configured with its own PostgreSQL database in docker-compose.yml.

• RabbitMQ is exposed on ports 5672 and 15672 for AMQP and the management interface.

• The API gateway is exposed on port 8080.

• The service-to-service network is named event_driven_commerce.

• Database initialization scripts are provided under configs/.

5. Architecture Screenshot

The following is the architecture diagram supplied inside the project itself. It shows the platform UI communicating with the API gateway, gRPC calls to customer/product/order services, PostgreSQL databases, and the RabbitMQ-based asynchronous payment path.

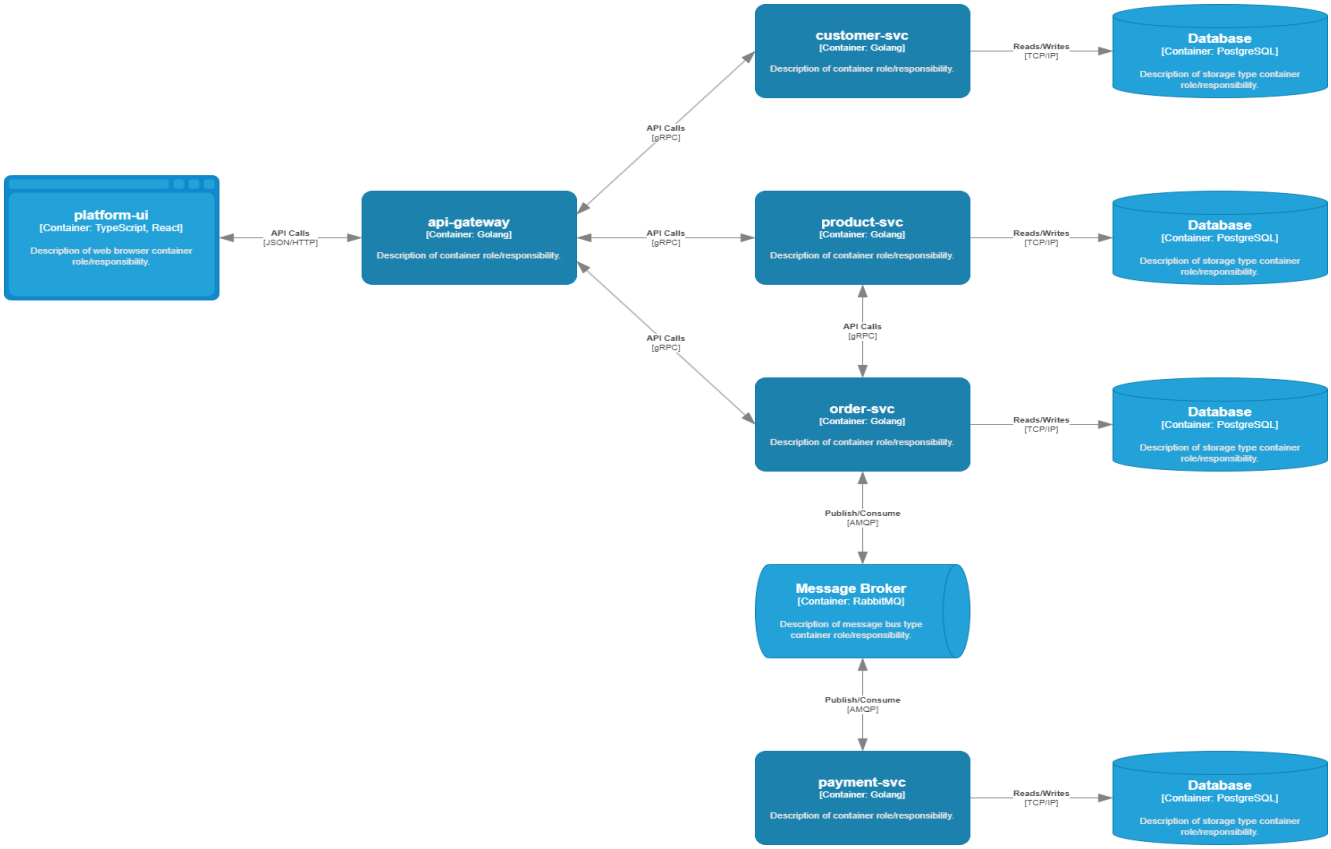


Figure 1 — Supplied event-driven commerce architecture diagram.

6. Project Structure Screenshot

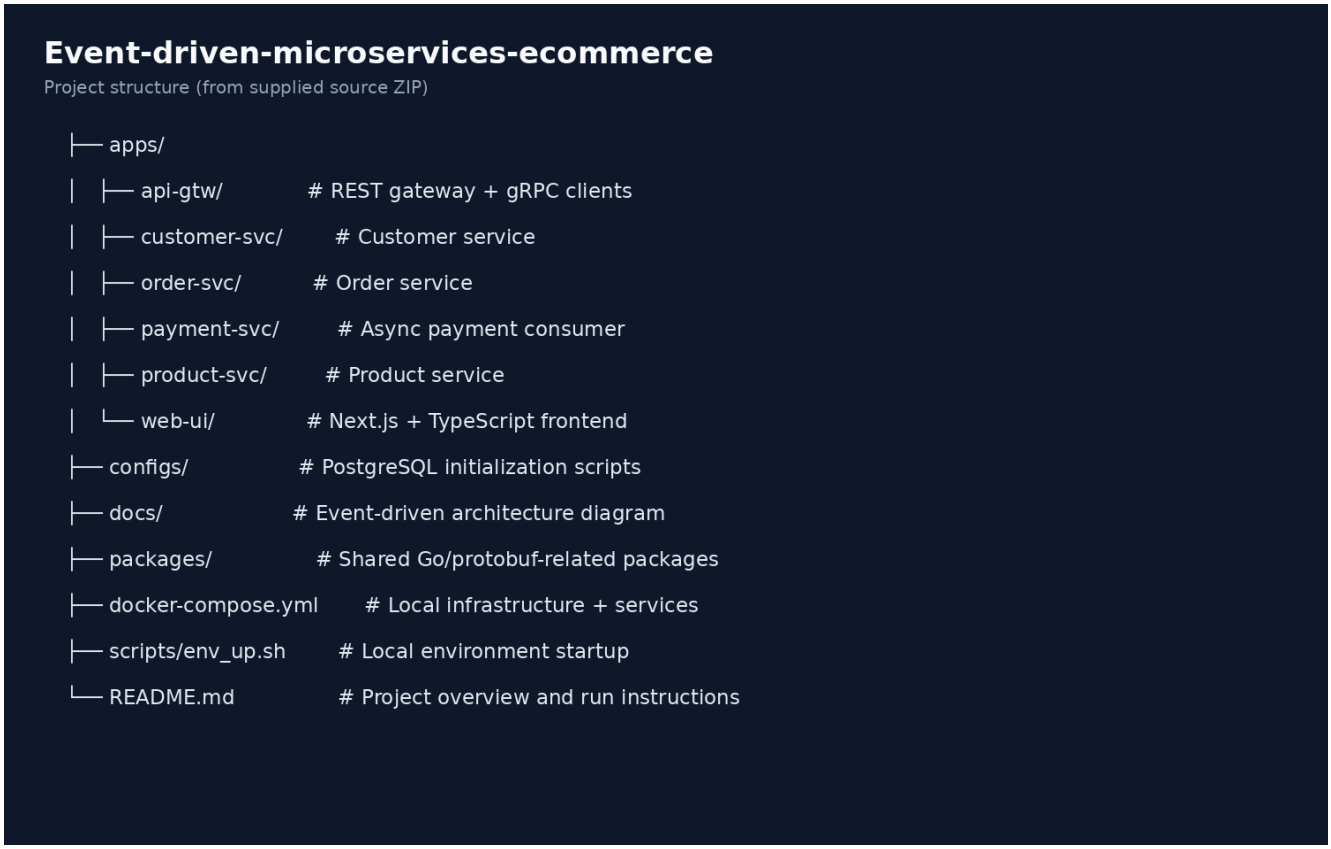


Figure 2 — Project structure derived from the supplied source ZIP.

7. API Gateway Source Screenshot

The gateway layer contains HTTP routing/controller code and gRPC clients, matching the README's description of REST endpoints for the web UI and internal gRPC communication.



Figure 3 — api-gtw router source from the supplied project.

8. Frontend Source Screenshot

The web-ui application is a Next.js frontend written in TypeScript. The supplied source includes authentication, product, cart, checkout and order routes/components.

apps/web-ui/src/app/page.tsx

```tsx
import Link from "next/link";

import { Button } from "@/components/ui/button";
import Image from "next/image";

export default async function Home() {
  return (
    <main className="flex flex-col items-center">
      <div className="px-10 py-28 flex flex-col md:flex-row justify-evenly gap-8 w-full">
        <div className="flex flex-col gap-8">
          <h1 className="text-3xl font-bold tracking-tighter sm:text-5xl xl:text-6xl/none">
            Shop the Latest Trends
          </h1>
          <p className="max-w-[400px] text-zinc-800 dark:text-zinc-300 md:text-xl">
            Discover our hand-picked selection of high-quality products.
          </p>
          <Button className="w-36">
            <Link href="/product">View products</Link>
          </Button>
        </div>

        <Image src="/home.svg" alt="home" width={300} height={200} />
      </div>
    </main>
  );
}
```

Figure 4 — Next.js web UI page source from the supplied project.

9. Run Instructions

The project README states that Docker and docker-compose are required for the local environment. The provided startup script is:

```
sh scripts/env_up.sh
```

The docker-compose configuration starts RabbitMQ, four PostgreSQL databases, the API gateway, customer service, product service, order service and payment service on a shared Docker network.

10. Request & Event Flow

• Customer uses the Next.js web UI.

• Web UI sends REST/HTTP requests to api-gtw.

• api-gtw calls customer-svc, product-svc or order-svc over gRPC as needed.

• Order creation uses order-svc and product-service communication.

• For payment processing, order-svc publishes an asynchronous message to RabbitMQ.

• payment-svc consumes the message and processes the payment workflow.

• Each service persists its own data in its corresponding PostgreSQL database.

11. Conclusion

The supplied project demonstrates a distributed e-commerce system using independently deployable Go services, synchronous gRPC communication, a REST API gateway, separate PostgreSQL databases and asynchronous RabbitMQ messaging for payment processing. The included architecture diagram and source-structure screenshots provide visual documentation of the implementation.